

# Technical Documentation: Fetch & Extract Prototype

Project: fetch\_extract\_v1 | Date: 2026-01-07

## Table of Contents

### 1. Project Overview

### 2. Architecture

2.1. High-Level Flow

2.2. Component Diagram

### 3. File & Module Details

3.1. ssrf.py - SSRF Protection Module

3.2. extract.py - Text Extraction Module

3.3. fetcher.py - Page Fetching Module

3.4. api.py - FastAPI Endpoints

3.5. example\_client.py - Demo Client

### 4. Testing & Validation

4.1. Test Summary

4.2. How to Run Tests

### 5. API Test Results (Swagger)

5.1. Tested URLs & Extraction Samples

5.2. SSRF Blocking Verification

### 6. Requirements Satisfaction

### 7. Dependencies

### 8. How to Use

### 9. Future Considerations

# 1. Project Overview

The **Fetch & Extract Prototype** ( `fetch_extract_v1` ) is a specialized service designed to safely fetch web page content and convert it into a deterministic, clean text format. Its primary purpose is to support the Research Agent platform by providing a reliable mechanism for evidence referencing.

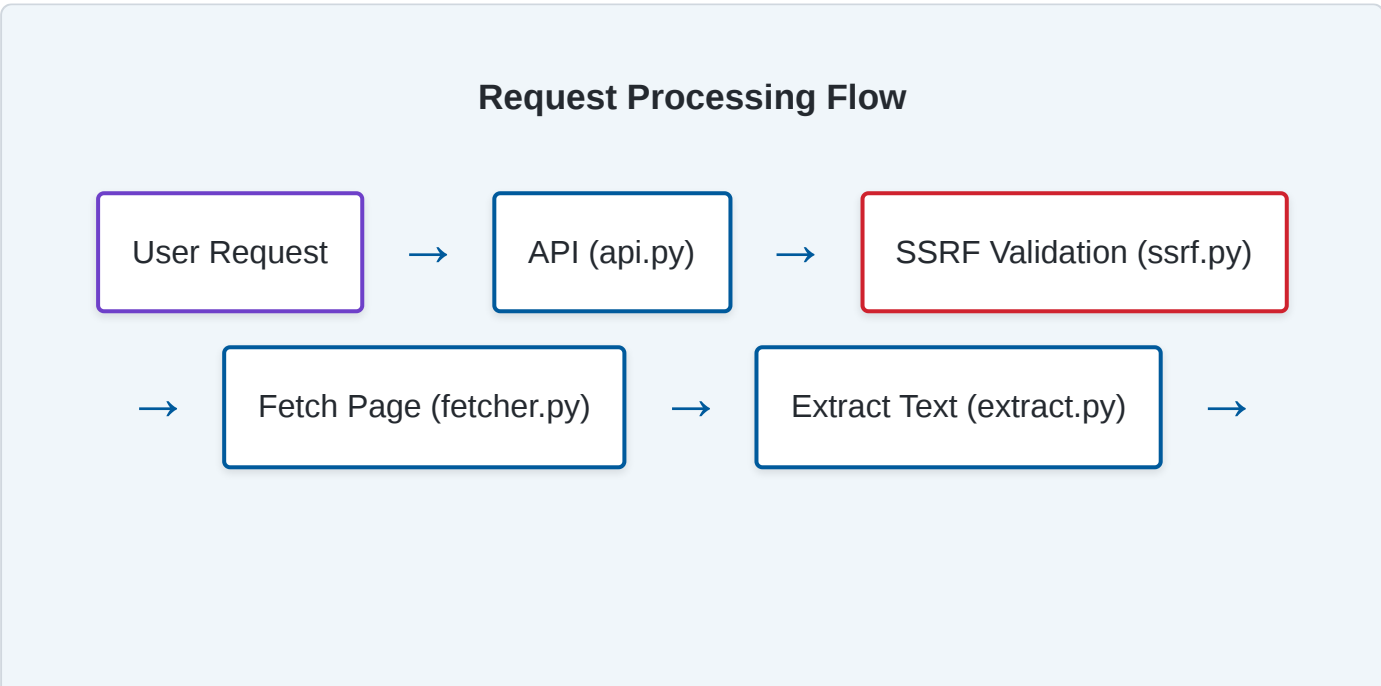
The core problem this prototype solves is the need for stable character offsets in web content. For the Research Agent to create durable citations, such as `"Price: $19.99 at characters 802-817"` , the underlying text and its character positions must not change between fetches of the same HTML content. This prototype guarantees stability through a deterministic extraction pipeline and provides content hashes for change detection.

- **Project Name:** Fetch & Extract Prototype (`fetch_extract_v1`)
- **Location:** `src/platform/prototypes/fetch_extract_v1/`
- **Branch:** `feature/research-platform-fetch-extract`

## 2. Architecture

### 2.1. High-Level Flow

A user request to fetch a URL initiates a sequential, security-first process. Each step must complete successfully before the next is initiated, ensuring that unsafe URLs are rejected early and that fetched content is processed consistently.



Return Response

## 2.2. Component Diagram

The prototype is composed of several distinct Python modules, each with a specific responsibility. The `api.py` module serves as the entry point, orchestrating calls to the security, fetching, and extraction layers.

### fetch\_extract\_v1 Component Architecture

### api.py (FastAPI Endpoints)

- `GET /health`: Health check endpoint.
- `POST /v1/fetch`: Primary endpoint to fetch and extract text from a URL.



### ssrf.py (Security Layer)

- URL validation and scheme checking (HTTP/HTTPS only).
- DNS resolution and blocking of private/reserved IP ranges.
- Enforcement of a domain allowlist.
- Recursive validation of all redirect destinations.



### fetcher.py (Page Fetching)

- Uses Playwright for robust, headless browser automation (Chromium).
- Retrieves final HTML content after JavaScript rendering.
- Handles HTTP response codes and content types.



### extract.py (Text Extraction)

- Parses HTML using BeautifulSoup with the deterministic `lxml` parser.
- Traverses the DOM in a fixed order to ensure consistency.
- Normalizes whitespace and formats content (headings, lists, tables).



### Content Hash (SHA256)

- A SHA256 hash is computed from the final `clean_text` output.
- This hash serves as a version identifier for the extracted content.

# 3. File & Module Details

## 3.1. ssrf.py - SSRF Protection Module

This module is the first line of defense, designed exclusively to prevent Server-Side Request Forgery (SSRF) attacks. It validates every URL before any network request is made.

### Key Functions:

- `validate_url_ssrf(url)`: The main validation function that orchestrates all security checks.
- `is_private_ip(ip)`: Checks a given IP address against a comprehensive list of private, loopback, link-local, and reserved ranges (both IPv4 and IPv6).
- `resolve_dns(hostname)`: Safely resolves a hostname to its IP address(es) and checks each one for safety.

### Security Checks Pipeline:

| Check               | Description                                                                                                                                                          |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scheme Validation   | Ensures the URL scheme is either <code>http://</code> or <code>https://</code> .                                                                                     |
| DNS Resolution      | The hostname is resolved to an IP address <i>before</i> the fetcher attempts to connect. This prevents DNS rebinding attacks.                                        |
| IP Blocking         | Blocks all IPs in private ranges (e.g., <code>127.0.0.1</code> , <code>192.168.x.x</code> , <code>10.x.x.x</code> , <code>172.16-31.x.x</code> , <code>::1</code> ). |
| Domain Allowlist    | The URL's domain must be present in a centrally managed allowlist located at <code>app/core/domains.py</code> .                                                      |
| Redirect Validation | The fetcher is configured to pass each redirect location back to this module for full validation before following it.                                                |

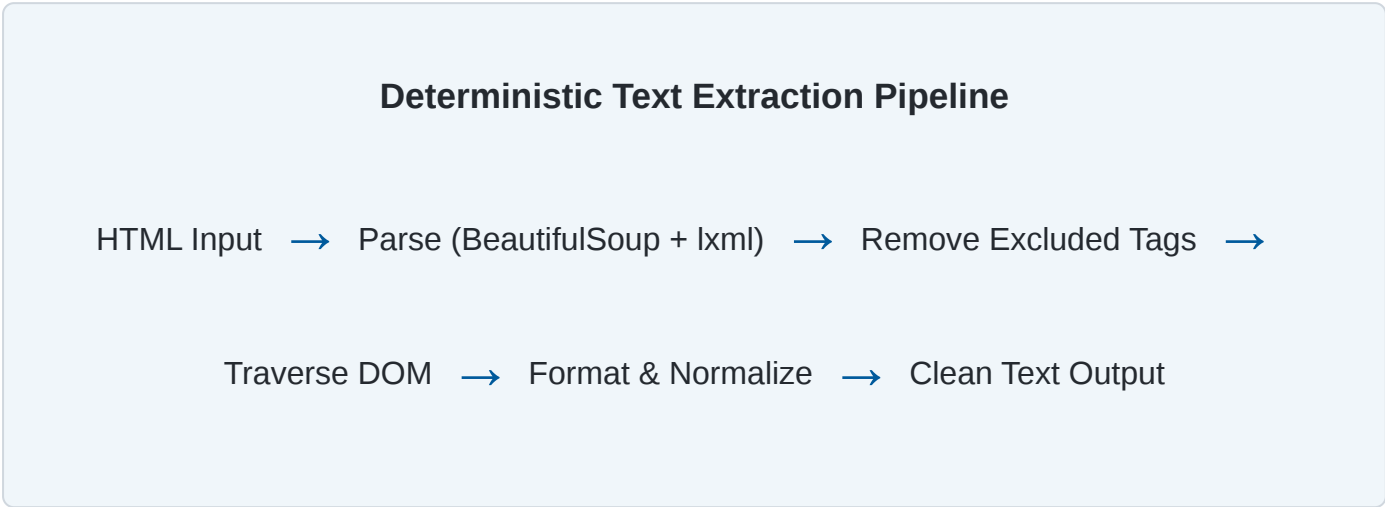
## 3.2. extract.py - Text Extraction Module

The core of the content processing logic resides here. This module's sole purpose is to convert raw HTML into a structured, deterministic plain text string, which is critical for stable offset calculations.

### Key Functions:

- `extract_html_to_text(html, options)`: The main entry point that takes raw HTML and returns the clean text string.
- `ExtractOptions`: A Pydantic/dataclass object for configuring extraction behavior (e.g., which tags to exclude).

### Extraction Pipeline:



### Output Format Example:

The extractor produces a markdown-like format for readability and structure.

```
TITLE: Page Title
# Main Heading
This is a paragraph of text.

## Subheading
- This is a bullet item.
- This is another bullet item.

TABLE:
```

## Determinism Guarantees:

---

- **Fixed Parser:** Always uses the `lxml` parser, which is known for its consistent parsing behavior.
- **Ordered Traversal:** The DOM is always traversed in document order.
- **Consistent Whitespace Rules:** Whitespace is normalized according to a strict, predefined set of rules.
- **Volatile Element Removal:** Elements like `<script>`, `<style>`, `<nav>`, and `<footer>` are removed before processing to avoid including non-content text.

## 3.3. fetcher.py - Page Fetching Module

---

This module handles the interaction with the web, using Playwright to control a headless Chromium browser. This approach ensures that pages heavily reliant on JavaScript for rendering are processed correctly.

### Key Functions:

---

- `Fetcher.fetch(url)`: Fetches the content of a single URL, returning the final HTML, status code, and other metadata.
- `compute_content_hash(text)`: A utility function to compute the SHA256 hash of the extracted text.
- `crawl_site(url, max_pages, max_depth)`: An experimental function for multi-page crawling (not exposed via the primary API endpoint).

## 3.4. api.py - FastAPI Endpoints

---

This module exposes the prototype's functionality via a RESTful API using the FastAPI framework. It includes request/response models for type safety and automatic generation of interactive documentation (Swagger UI).

### Endpoints:

---

| Method | Path                       | Description                                                                  |
|--------|----------------------------|------------------------------------------------------------------------------|
| GET    | /health                    | Simple health check, returns <code>{"status": "ok"}</code> .                 |
| POST   | /v1/fetch                  | The main endpoint. Accepts a URL and returns extracted content.              |
| POST   | /v1/fetch/evidence-example | A demo endpoint that fetches a predefined URL and shows an evidence example. |
| GET    | /docs                      | Auto-generated Swagger UI for interactive API exploration.                   |

**Request Schema ( `POST /v1/fetch` ):**

```
{
  "url": "https://example.com"
}
```

**Response Schema:**

```
{
  "source_url": "https://example.com",
  "final_url": "https://example.com",
  "fetched_at": "2026-01-07T00:00:00Z",
  "status_code": 200,
  "content_type": "text/html",
  "clean_text": "TITLE: Example\n# Heading\nContent...",
  "clean_text_length": 1234,
  "content_hash": "sha256:abc123def456...",
  "extraction_version": "cleantext_v1",
  "title": "Example",
  "links_count": 50,
  "error": null
}
```

### 3.5. example\_client.py - Demo Client

A command-line script that demonstrates the end-to-end workflow of using the API to generate and verify evidence. It serves as a practical example for integrating services.



## Workflow:

1. Sends a request to the `/v1/fetch` endpoint for a target URL.
2. Receives the response and programmatically finds specific text snippets (e.g., a price or a name).
3. Calculates the character start and end offsets for these snippets within the `clean_text`.
4. Stores the evidence (snippet, offsets, `content_hash`) in a local file, `evidence_store.json`.
5. Demonstrates how to re-fetch and verify that the offsets remain valid if the `content_hash` is unchanged.

## 4. Testing & Validation

The prototype is covered by a suite of `pytest` tests to ensure correctness, security, and stability. All 22 tests are currently passing.

### 4.1. Test Summary

- **Test Files:** `Tests/test_offset_stability.py` (18 tests), `Tests/test_modules.py` (4 tests)
- **Total Tests:** 22
- **Result:** 22 PASSED 

| Category         | Tests | What They Verify                                                                                           |
|------------------|-------|------------------------------------------------------------------------------------------------------------|
| Offset Stability | 5     | Ensures that identical HTML input produces bit-for-bit identical text output over 100 consecutive runs.    |
| Known Offsets    | 3     | Verifies that the character offsets of known text snippets on test pages match hardcoded, expected values. |
| Deduplication    | 2     | Checks that repeated text blocks are handled consistently without affecting offsets of subsequent content. |
| SSRF Blocking    | 4     | Confirms that requests to private/disallowed IPs and domains are blocked, while valid URLs are allowed.    |

|                 |   |                                                                                                                       |
|-----------------|---|-----------------------------------------------------------------------------------------------------------------------|
| Extract Options | 2 | Validates that changing extraction options (e.g., excluding different tags) produces the expected, consistent output. |
| Whitespace      | 2 | Asserts that complex whitespace scenarios are normalized into a canonical form.                                       |

## 4.2. How to Run Tests

To execute the test suite, set the `PYTHONPATH` to include the project root and then run `pytest`.

```
# Set the PYTHONPATH to include the project's root directory
$env:PYTHONPATH = "PROJECT_ROOT;PROJECT_ROOT/src/platform/prototypes/fetch_extract_v1"

# Run pytest with verbose output
python -m pytest src/platform/prototypes/Tests/ -v
```

## 5. API Test Results (Swagger)

The API was tested against a variety of real-world websites to validate its fetching and extraction capabilities. The results confirm that the prototype can handle diverse HTML structures and content types.

### 5.1. Tested URLs & Extraction Samples

- ▶ 1. Books to Scrape ([books.toscrape.com](https://books.toscrape.com))
- ▶ 2. Wikipedia - SEO ([en.wikipedia.org/wiki/Search\\_engine\\_optimization](https://en.wikipedia.org/wiki/Search_engine_optimization))
- ▶ 3. GitHub Trending ([github.com/trending](https://github.com/trending))
- ▶ 4. Amazon.com ([www.amazon.com](https://www.amazon.com))

## 5.2. SSRF Blocking Verification

The SSRF protection layer was explicitly tested by attempting to fetch disallowed URLs. All attempts were correctly blocked by the `ssrf.py` module before any network request was made by the fetcher.

- `http://localhost/` - **✗ BLOCKED** (Loopback IP)
- `http://192.168.1.1/` - **✗ BLOCKED** (Private IP)
- `https://10.0.0.1/` - **✗ BLOCKED** (Private IP)
- `https://example.com/` - **✗ BLOCKED** (Domain not in allowlist)

## 6. Requirements Satisfaction

The prototype successfully meets all initial project requirements.

| Requirement             | Status | Implementation Method                                               |
|-------------------------|--------|---------------------------------------------------------------------|
| SSRF-safe Fetching      | ✓      | <code>ssrf.py</code> module blocks disallowed URLs before fetching. |
| HTTP/HTTPS Only         | ✓      | Scheme validation in <code>ssrf.py</code> .                         |
| Block Private IPs       | ✓      | IP range checking against a blocklist in <code>ssrf.py</code> .     |
| Domain Allowlist        | ✓      | Uses a shared configuration from <code>app/core/domains.py</code> . |
| Clean Text Extraction   | ✓      | The <code>extract.py</code> pipeline converts HTML to clean text.   |
| Title Extraction        | ✓      | Extracted and prefixed with <code>TITLE:</code> .                   |
| Headings Extraction     | ✓      | Formatted with <code>#</code> and <code>##</code> prefixes.         |
| Visible Text Extraction | ✓      | Extracts all text from non-excluded block and inline elements.      |
| Table Text Extraction   | ✓      | Identified and prefixed with <code>TABLE:</code> .                  |
| Bullet Lists Extraction | ✓      | List items are prefixed with <code>-</code> .                       |
| Stable Offsets          | ✓      | Guaranteed by the deterministic extraction pipeline.                |

|                   |   |                                                                     |
|-------------------|---|---------------------------------------------------------------------|
| Content Hash      | ✓ | SHA256 hash of <code>clean_text</code> is generated for versioning. |
| Integration Tests | ✓ | A comprehensive suite of 22 tests validates all key features.       |

## 7. Dependencies

The prototype relies on the following third-party Python libraries. These should be included in a `requirements.txt` file.

```
fastapi>=0.100.0
uvicorn>=0.23.0
playwright>=1.40.0
beautifulsoup4>=4.12.0
lxml>=5.0.0
pydantic>=2.0.0
pydantic-settings>=2.0.0
httpx>=0.25.0
pytest>=7.4.0
```

## 8. How to Use

Follow these steps to run the API server and make a request.

### 1. Start the API Server

Navigate to the project's root directory and run the `api.py` module. Ensure the `PYTHONPATH` is set correctly.

```
cd PROJECT_ROOT
$env:PYTHONPATH = "PROJECT_ROOT;PROJECT_ROOT/src/platform/prototypes/fetch_extract_v1"
python src/platform/prototypes/fetch_extract_v1/api.py
```

### 2. Access the Swagger UI

Once the server is running, open a web browser and navigate to the following URL to access the interactive API documentation:

### 3. Fetch a URL via cURL

---

Alternatively, you can use a command-line tool like `cURL` to send a POST request to the `/v1/fetch` endpoint.

```
# Example for PowerShell
$body = '{"url": "https://books.toscrape.com"}'
curl -X POST "http://localhost:8000/v1/fetch" -H "Content-Type: application/json" -d $body

# Example for Bash
curl -X POST "http://localhost:8000/v1/fetch" \
-H "Content-Type: application/json" \
-d '{"url": "https://books.toscrape.com"}'
```

## 9. Future Considerations

---

Upon successful validation, the following enhancements and integrations are planned:

1. **Codebase Integration:** Move the prototype from its sandboxed location into the main application codebase.
2. **Caching Layer:** Implement a caching mechanism (e.g., Redis) to store results for recently fetched URLs, reducing redundant processing and external network calls.
3. **Rate Limiting:** Add a rate-limiting policy to the API to prevent abuse and manage load.
4. **Expanded Extraction Options:** Enhance `extract.py` to support the extraction of other content types, such as image URLs/alt-text and structured metadata (e.g., JSON-LD).
5. **Research Agent Integration:** Fully integrate the service with the Research Agent's evidence storage and citation generation system.